



POLITECNICO DI TORINO

MASTER'S DEGREE IN CYBERSECURITY ENGINEERING

Design and Implementation of an eBPF-based Security Monitoring Tool

Supervisor:

Prof. Riccardo Sisto

Candidate:

Simone Romano

Company Tutors:

Dr. David Soldani

Dr. Vincenzo Cosi

Academic Year 2025-2026

This thesis presents the design and implementation of a security monitoring tool that collects and analyzes process-related information directly from the operating system kernel using eBPF (extended Berkeley Packet Filter). By leveraging kernel-level instrumentation, the system captures detailed process events with high accuracy and minimal overhead, overcoming the limitations of traditional user-space monitoring approaches.

The proposed tool adopts a modular and efficient architecture to ensure scalability and real-time data collection. The gathered information is structured to support security analysis, enabling the detection of anomalous behaviors such as unauthorized executions and privilege escalations.

This work demonstrates how kernel-derived process data, combined with eBPF technology, can enhance system visibility and provide a solid foundation for advanced security monitoring and threat detection.

Acknowledgements

Only in the printed version

Contents

Acronyms	1
1 Introduction	2
1.1 Context and Motivation	2
1.1.1 Runtime Security and Kernel-Level Visibility	2
1.1.2 eBPF as an Enabling Technology	3
1.1.3 Industrial and Deployment Context	3
1.2 Problem Statement	4
1.2.1 Security-Relevant Process Monitoring	4
1.2.2 Technical Constraints	5
1.3 Objectives and Research Questions	5
1.3.1 Main Objective	5
1.3.2 Specific Objectives	5
1.3.3 Research Questions	6
1.4 Contributions of the Work	6
1.4.1 System and Implementation Contributions	6
1.4.2 Detection and Security Contributions	7
1.4.3 Distinctive Aspects of the Proposed Approach	7
1.5 Scope and Limitations	8
1.5.1 In-Scope Capabilities	8
1.5.2 Out-of-Scope Capabilities	8

CONTENTS

1.6 Methodology	9
1.7 Thesis Structure	9
A Appendix A	10
B Appendix B	11
Bibliography	12

List of Figures

List of Tables

Acronyms

eBPF extended Berkeley Packet Filter

Chapter 1

Introduction

1.1 Context and Motivation

1.1.1 Runtime Security and Kernel-Level Visibility

Modern Linux systems execute applications, containers, and infrastructure services whose security-relevant behaviour changes continuously at runtime. Static inspection can describe software before execution, while application logs expose only the information that applications choose to report. Neither source alone provides a complete account of operations such as process creation, credential changes, file access, memory protection changes, namespace activity, or interactions with kernel security mechanisms. Runtime security monitoring therefore requires an observation point that is close enough to the operating system to capture these actions as they occur.

The Linux kernel is a valuable observation layer because it mediates processes, files, credentials, memory, and other shared resources. Kernel-level telemetry can reveal behaviour that is distributed across several applications or omitted from their logs. This visibility is particularly relevant in containerised environments, where workloads remain isolated at the application level but share the host kernel. At the same time, kernel instrumentation is security-sensitive: excessive privileges or unsafe extensions may increase the attack surface, especially across container boundaries [6]. A monitoring design must consequently balance visibility, compatibility, operational cost, and the risks associated with privileged execution.

Runtime security tools increasingly use eBPF to obtain this visibility. Ex-

isting systems demonstrate that eBPF can support process, file, network, and security telemetry in cloud-native environments, although their architectures and performance characteristics differ [7]. This thesis focuses on the process and host-security side of that design space.

1.1.2 eBPF as an Enabling Technology

eBPF provides a controlled runtime inside the Linux kernel. User-space software loads constrained programs that can be attached to supported tracing, networking, and security hooks. Before a program is accepted, the kernel verifier analyses its control flow and use of memory, registers, pointers, and helper functions. Accepted programs can then collect or transform data when the corresponding hook is reached [5, 8, 9].

This execution model is well suited to runtime monitoring for three reasons. First, collection occurs at the point where the kernel processes the operation, reducing dependence on application-specific instrumentation. Second, eBPF maps and event buffers provide structured communication between kernel and user space. Third, programs can reject irrelevant activity before transporting it to user space, which creates an opportunity to control processing overhead. These properties do not make an eBPF tool automatically safe, portable, or efficient: verifier behaviour, available program types, helper functions, kernel data structures, and attachment mechanisms vary across kernel versions. They instead provide the foundation on which a target-aware monitoring pipeline can be built.

1.1.3 Industrial and Deployment Context

The work was developed in an industrial collaboration and targets Rocky Linux 8.10 with the RHEL-compatible kernel `4.18.0-553.109.1.el8_10.x86_64`. This environment is important because it combines a long-lived enterprise kernel line with backported functionality. A feature exposed by a newer upstream kernel cannot be assumed to exist, or to behave identically, on this target. Compatibility is therefore treated as an architectural constraint rather than as a final packaging concern.

The resulting prototype, named *Vesuvius*, is intended to operate as a node-level monitoring agent. Its container image packages both the compiled executable and the eBPF object, and its intended Kubernetes deployment model places one privileged monitoring instance on each selected node. Ku-

bernetes DaemonSets are designed for this class of node-scoped service and are commonly used for monitoring and logging agents [11]. Cluster-wide ingestion and correlation may be performed by an external platform; Vesuvius itself is deliberately concerned with bounded local collection and detection. The thesis does not claim that a complete Kubernetes deployment has been validated.

1.2 Problem Statement

1.2.1 Security-Relevant Process Monitoring

The central problem addressed by this thesis is how to convert heterogeneous kernel observations into useful security information on a constrained enterprise Linux target. Capturing an event is only the first step. A practical pipeline must assign a stable identity to the event, transport it without confusing records produced by different probes, decode typed arguments, apply user-selected scope, and present the result consistently. When these contracts diverge, valid kernel records may be decoded as unknown events, attributed to the wrong event type, or silently discarded.

Individual observations also have different analytical value. Some are relevant in isolation, for example an access to a sensitive file under narrowly defined conditions. Others become relevant only as an ordered group, such as a credential transition followed shortly by process execution or sensitive file access. Following the established anomaly-detection taxonomy, the thesis uses *point event* for a locally significant observation and *short collective event pattern* for a bounded sequence whose combined meaning is more relevant than its elements considered independently [4]. These terms describe deterministic, rule-based detection; they do not imply machine learning, statistical baselines, or unsupervised anomaly detection.

Policies and detectors have distinct roles in the proposed system. Policies select the event domain that should be collected and evaluated. Detectors express conditions over those normalised events and may produce an alert from one event or from a short process-aware sequence. Alerts carry MITRE ATT&CK metadata to classify and communicate the observed behaviour. ATT&CK organises adversary behaviour through tactics and techniques [10]; the mapping used by Vesuvius provides traceability, but does not by itself prove detector correctness, coverage, or novelty.

1.2.2 Technical Constraints

Four constraints shape the problem. First, the target kernel limits which attachment mechanisms, helpers, maps, and portability techniques can be used reliably. Second, the kernel verifier constrains program complexity and memory access, so event enrichment must be divided carefully between kernel and user space. Third, high-frequency hooks can generate enough data to make transport, decoding, rule evaluation, and output formatting more expensive than the probe itself. Fourth, the agent is expected to run beside production workloads; CPU and memory use are therefore evaluation criteria rather than secondary implementation details.

These constraints motivate a selective architecture. Vesuvius uses eBPF for collection and optional early filtering, while user space owns event normalisation, policy management, detector evaluation, alert generation, and presentation. The design preserves a stable kernel-to-user-space contract while allowing detection logic to evolve without recompiling every eBPF program.

1.3 Objectives and Research Questions

1.3.1 Main Objective

The main objective of this thesis is to design, implement, and evaluate an eBPF-based runtime security monitoring prototype that collects process and security-relevant kernel events on the target Rocky Linux environment, normalises them in user space, and evaluates them through configurable policies and local detectors.

1.3.2 Specific Objectives

The work pursues the following specific objectives:

- collect selected process and security events through eBPF programs compatible with the target kernel;
- define a coherent contract across eBPF event identifiers, transport buffers, the Go decoder, the event registry, and output components;

- support declarative event selection through policies and local rule-based detection through YAML detector definitions;
- detect both security-relevant point events and bounded collective patterns using limited process-aware correlation;
- associate detector output with MITRE ATT&CK metadata for classification and downstream traceability;
- evaluate functional behaviour, CPU consumption, memory use, and the effect of an experimental early kernel-side user-identifier filter.

1.3.3 Research Questions

The thesis is guided by three research questions:

- RQ1** Which architectural and implementation adaptations are required to build an eBPF-based process and security monitoring pipeline on the target Rocky Linux 8.10 kernel?
- RQ2** How can heterogeneous kernel events be normalised and evaluated through declarative policies and detectors to identify security-relevant point events and short collective event patterns?
- RQ3** What CPU and memory overhead does the proposed pipeline introduce under the selected benchmark profiles, and how does early kernel-side user-identifier filtering affect that overhead?

1.4 Contributions of the Work

1.4.1 System and Implementation Contributions

The first contribution is a target-aware monitoring architecture adapted to Rocky Linux 8.10. It combines selected process and security probes with a Go runtime responsible for probe attachment, event transport, decoding, filtering, and structured output. The implementation covers security-relevant areas including process execution and lifecycle, credential transitions, file operations, memory activity, namespaces, capabilities, signalling, and selected kernel-integrity operations. This is selected coverage, not a claim of

complete Linux observability or feature equivalence with broader runtime-security products.

The second contribution is a coherent event contract across kernel and user space. Event identifiers and probe dependencies are represented explicitly so that user-selectable events are distinguished from internal attachment dependencies. This separation addresses a concrete failure mode observed during development: multiple programs may share transport maps, yet only the programs required by the selected events should be attached and only their corresponding records should be exposed as public events. Typed decoding, argument enrichment, and separate outputs for diagnostics, raw events, and alerts complete this processing path.

1.4.2 Detection and Security Contributions

The third contribution is a declarative local detection pipeline. Policies select relevant event sets, while YAML detector definitions describe point conditions or ordered steps. The runtime evaluates these definitions, keeps bounded state for short collective patterns, applies limited process-aware grouping, suppresses repeated alerts where configured, and emits structured evidence. Detector definitions also propagate MITRE ATT&CK tactics and techniques into alert output. Declarative policies and multi-event detection are established ideas in systems such as Tracee [3, 1]; the contribution here lies in their integration with the Vesuvius event model and target constraints, not in claiming these ideas in isolation.

The fourth contribution concerns operational and performance controls. Vesuvius provides configurable event and command filters, alert-only output, structured runtime logging, reproducible benchmark profiles, and container packaging for the executable and embedded eBPF object. An experimental kernel-side user-identifier filter can discard events before transport. Its value is assessed empirically in this thesis; the design objective of remaining below five percent of one CPU core is not presented as an achieved result.

1.4.3 Distinctive Aspects of the Proposed Approach

The distinctive aspect of the work is the design and evaluation of a lightweight, bounded, and process-aware local detection pipeline integrated with an eBPF monitoring agent adapted to the constraints of Rocky Linux 8's 4.18-based enterprise kernel. The approach intentionally limits local state and

temporal windows because the agent is expected to run on monitored nodes with a constrained resource budget. It produces point alerts and short collective evidence locally, while leaving cluster-wide or long-lived contextual reasoning to an external system with broader visibility.

This aspect is treated as a candidate contribution whose significance must be supported by comparison and evaluation in later chapters. eBPF monitoring, YAML configuration, MITRE ATT&CK tagging, kernel filtering, and multi-event detection are not individually claimed as novel. Tracee is used as a principal engineering reference because it documents a mature eBPF event and policy architecture [2]; Vesuvius adopts relevant design patterns selectively and changes them where the target kernel, reduced scope, or bounded local-detection model requires a different trade-off.

1.5 Scope and Limitations

1.5.1 In-Scope Capabilities

The thesis covers node-level collection of selected Linux process and security events, their transport and normalisation in user space, declarative policy selection, rule-based point detection, and bounded collective detection. It also covers structured event and alert output, MITRE ATT&CK metadata propagation, container packaging, functional tests, and performance measurements on the specified Rocky Linux environment. Selected networking instrumentation exists as an adjacent, collaboratively developed capability, but it is not the main contribution or evaluation focus of this work.

1.5.2 Out-of-Scope Capabilities

Vesuvius is a research prototype rather than a production-ready prevention platform. Active enforcement, automatic response, machine-learning-based anomaly detection, distributed correlation across nodes, and cluster-contextual detection are outside the scope of the implemented local engine. The work does not claim universal kernel portability, exhaustive event coverage, equivalence with Tracee, semantic validation of ATT&CK mappings, or validated production operation in Kubernetes. Likewise, the performance target is evaluated under defined profiles and is not assumed to hold for every workload, event selection, or output configuration.

1.6 Methodology

The research follows an iterative design, implementation, and evaluation methodology. The target environment and security-monitoring requirements are first translated into an architecture that separates kernel collection from user-space interpretation. Tracee source code and documentation are examined as an engineering reference, while each adopted pattern is checked against the capabilities of the target kernel. Features are then implemented incrementally across the eBPF programs, probe registry, decoder, policy manager, detector engine, and output layer.

Verification combines automated Go tests, build-time checks for the eBPF object and embedded binary, and targeted runtime tests that trigger individual hooks or detector sequences. Registry and decoding consistency are checked end to end because successful probe attachment alone does not demonstrate correct event interpretation. Performance is evaluated through repeatable profiles that record CPU utilisation, resident memory, and thread count after a warm-up period. Filtered and unfiltered configurations are compared under the same event and detector workload. This methodology supports an evidence-based answer to the research questions while keeping implementation detail in the design, implementation, and evaluation chapters.

1.7 Thesis Structure

The remainder of this thesis is organised as follows. Chapter 2 presents the technical background and related work, including Linux runtime instrumentation, eBPF, runtime-security tools, anomaly categories, and policy-driven detection. Chapter 3 derives the requirements and describes the architecture of Vesuvius. Chapter 4 explains the implementation of the kernel programs, user-space event pipeline, policy subsystem, detector engine, logging, and packaging. Chapter 5 evaluates functional correctness, event coverage, detector behaviour, and performance on the target environment. Chapter 6 summarises the results, answers the research questions, discusses limitations, and identifies directions for future work.

Appendix A

Appendix A

Appendix text here.

Appendix B

Appendix B

Appendix text here.

Bibliography

- [1] Aqua Security. *K8s API Connection*. Tracee v0.22 built-in signature documentation. URL: https://aquasecurity.github.io/tracee/v0.22/docs/events/builtin/signatures/kubernetes_api_connection/ (visited on 07/17/2026).
- [2] Aqua Security. *Tracee Documentation*. URL: <https://aquasecurity.github.io/tracee/latest/> (visited on 07/17/2026).
- [3] Aqua Security. *Tracee Policies: Overview*. URL: <https://aquasecurity.github.io/tracee/dev/docs/policies/> (visited on 07/17/2026).
- [4] Varun Chandola, Arindam Banerjee, and Vipin Kumar. “Anomaly Detection: A Survey”. In: *ACM Computing Surveys* 41.3 (July 2009), 15:1–15:58. DOI: [10.1145/1541880.1541882](https://doi.org/10.1145/1541880.1541882).
- [5] Bolaji Gbadamosi et al. “The eBPF Runtime in the Linux Kernel”. In: *arXiv preprint arXiv:2410.00026* (2024). DOI: [10.48550/arXiv.2410.00026](https://doi.org/10.48550/arXiv.2410.00026). URL: <https://arxiv.org/abs/2410.00026>.
- [6] Yi He et al. “Cross Container Attacks: The Bewildered eBPF on Clouds”. In: *32nd USENIX Security Symposium (USENIX Security 23)*. Anaheim, CA: USENIX Association, 2023, pp. 5971–5988. ISBN: 978-1-939133-37-3. URL: <https://www.usenix.org/conference/usenixsecurity23/presentation/he>.
- [7] Jin Her et al. “An In-Depth Analysis of eBPF-Based System Security Tools in Cloud-Native Environments”. In: *IEEE Access* 13 (2025), pp. 155588–155604. DOI: [10.1109/ACCESS.2025.3605432](https://doi.org/10.1109/ACCESS.2025.3605432). URL: <https://ieeexplore.ieee.org/document/11146725/>.
- [8] Linux Kernel Community. *eBPF Userspace API*. Linux kernel 6.9 documentation. URL: <https://www.kernel.org/doc/html/v6.9/userspace-api/ebpf/index.html> (visited on 07/17/2026).

BIBLIOGRAPHY

- [9] Linux Kernel Community. *eBPF Verifier*. URL: <https://docs.kernel.org/bpf/verifier.html> (visited on 07/17/2026).
- [10] Blake E. Strom et al. *MITRE ATT&CK: Design and Philosophy*. Tech. rep. The MITRE Corporation, Mar. 2020. URL: <https://www.mitre.org/news-insights/publication/mitre-attck-design-and-philosophy>.
- [11] The Kubernetes Authors. *DaemonSet*. Kubernetes Documentation. URL: <https://kubernetes.io/docs/concepts/workloads/controllers/daemonset/> (visited on 07/17/2026).

To my family and friends, for their support.